

• List的使用

◦ 访问元素(索引)

```
fruits = ["apple", "banana", "cherry"]
print(fruits[0])    # 第一个元素
print(fruits[-1])   # 最后一个元素
```

• 添加元素

```
nums = [1, 2]
nums.append(3)        # 在末尾添加
nums.insert(1, 1.5)    # 在指定位置插入
print(nums)
# 输出: [1, 1.5, 2, 3]
```

• 删除元素

```
nums = [1, 2, 3, 4]
nums.remove(2)        # 删除指定值
nums.pop()            # 删除最后一个
del nums[0]           # 删除指定索引
print(nums)
# 输出: [3]
```

• 切片操作

```
matrix = [[1, 2], [3, 4], [5, 6]]
print(matrix[1][0])    # 访问第二行第一个元素
# 输出: 3, "e"
print(letters[1:4])     # 取索引1到3
print(letters[:3])      # 从开头到索引2
print(letters[::2])     # 每隔一个取一次
# 输出:
# ['b', 'c', 'd']
# ['a', 'b', 'c']
# ['a', 'c', 'e']
```

- 遍历列表

```
colors = ["red", "green", "blue"]
for c in colors:
    print(c)
# 输出:
# red
# green
# blue
```

- 合并与重复

```
a = [1, 2]
b = [3, 4]
print(a + b)      # 合并
print(a * 3)      # 重复
# 输出:
# [1, 2, 3, 4]
# [1, 2, 1, 2, 1, 2]
```

- 排序和反转

```
nums = [3, 1, 4, 2]
nums.sort()       # 升序排序
print(nums)       # [1, 2, 3, 4]
nums.reverse()    # 反转
print(nums)       # [4, 3, 2, 1]
```

- 列表推导式

```
squares = [x**2 for x in range(5)]
print(squares)
# 输出: [0, 1, 4, 9, 16]
```

- 列表的嵌套

```
matrix = [[1, 2], [3, 4], [5, 6]]
print(matrix[1][0]) # 访问第二行第一个元素
# 输出: 3
```

• Dict的使用

◦ 创建

```
# 直接创建
person = {"name": "Bob", "age": 20}

# 使用 dict() 函数
info = dict(city="Taipei", country="Taiwan")

# 空字典
empty = {}

print(person, info, empty)
# 输出: {'name': 'Bob', 'age': 20} {'city': 'Taipei', 'country': 'Taiwan'} {}
```

• 访问元素

```
student = {"name": "Alice", "age": 18}
print(student["name"]) # 通过键访问
print(student.get("grade")) # 使用 get(), 不存在不会报错
# 输出:
# Alice
# None
```

• 添加或修改元素

```
student = {"name": "Alice"}
student["age"] = 18 # 添加新键值对
student["name"] = "Bob" # 修改已存在的键
print(student)
# 输出: {'name': 'Bob', 'age': 18}
```

• 删除元素

```
student = {"name": "Alice", "age": 18, "grade": "A"}

student.pop("grade")    # 删除指定键
del student["age"]      # 删除指定键
student.clear()         # 清空字典

print(student)
# 输出: {}
```

• 字典的遍历

```
student = {"name": "Alice", "age": 18, "grade": "A"}

# 遍历键
for key in student:
    print(key)

# 遍历键和值
for key, value in student.items():
    print(key, value)
```

• 常用方法

`d.keys()` #获取键 `d.values()` #获取值 `d.items()` #获取键值对 `d.update()` #合并字典 `d.clear()` #清空字典

• 推导式

```
# 生成一个数字平方的字典
squares = {x: x**2 for x in range(3)}
print(squares)
# 输出: {0: 0, 1: 1, 2: 4}
```

• 嵌套字典

```
students = {
    "A01": {"name": "Alice", "age": 18},
    "A02": {"name": "Bob", "age": 19}
}
```

```
print(students["A01"]["name"])  
# 输出: Alice
```

• lambda匿名函数

◦ 语法

- lambda 参数:表达式
 1. 返回一个**函数对象**
 2. 常用于需要一个**临时函数**的场景(比如 map、filter、sorted 等)

◦ 用法

```
# 普通函数  
def add(x, y):  
    return x + y  
add_lambda = add(x,y)  
  
# 等价的 lambda 函数  
add_lambda = lambda x, y: x + y  
  
print(add(3, 5))          # 8  
print(add_lambda(3, 5))  # 8  
  
#进阶例子  
old_nums = [35, 12, 8, 99, 60, 52]  
new_nums = list(map(lambda x: x ** 2, filter(lambda x: x % 2 == 0,  
old_nums)))  
print(new_nums)  # [144, 64, 3600, 2704]
```

• Decorator装饰器

◦ 作用

- 本质上是一个**函数**，用来“包装”另一个函数，在不修改原函数代码的前提下，动态地添加额外功能。

◦ 基本语法

```
def decorator(func):  
    # 定义一个装饰器函数 decorator也可用log  
    # 等其他命名来说明此装饰器的作用
```

```
def wrapper(*args, **kwargs): # 内部再定义一个函数，用于包装被装饰的函数
    print("函数开始执行前...")
    result = func(*args, **kwargs) # 执行被装饰的函数
    print("函数执行结束后...")
    return result
    return wrapper # 返回包装后的函数

@decorator # 使用装饰器语法糖
def say_hello():
    print("Hello, Python!")

say_hello()
```

- 易错点

1. return wrapper
2. 不使用 *args,**kwargs
3. 多层装饰器,由下往上包裹

• 类Class、 Magic Methods

一、类 (Class) 是什么？

在 Python 中，**类** 是创建对象（实例）的模板，用来描述一类具有共同特征和行为的事物。类定义了对象的属性（变量）和方法（函数）。

• 二、类的基本语法

```
class 类名:
    def __init__(self, 参数):
        self.属性 = 参数

    def 方法名(self):
        # 执行动作
```

■ 示例：

```
class Person:
    def __init__(self, name, age):
        self.name = name # 实例属性
        self.age = age

    def say_hello(self):
```

```
        print(f"你好, 我叫 {self.name}, 今年 {self.age} 岁。")

# 创建对象
p1 = Person("小明", 18)
p1.say_hello()
```

输出：

你好, 我叫 小明, 今年 18 岁。

三、类中常见的概念

概念	含义
类 (Class)	模板、蓝图，用来创建对象。
对象 (Instance)	根据类创建的实际实体。
属性 (Attribute)	描述对象的特征（变量）。
方法 (Method)	描述对象的行为（函数）。
self	指向当前对象本身的引用。
__init__	构造函数，在创建对象时自动调用。

四、类属性与实例属性的区别

```
class Dog:
    kind = "犬科"    # 类属性（所有实例共享）

    def __init__(self, name):
        self.name = name    # 实例属性（每个对象不同）

dog1 = Dog("旺财")
dog2 = Dog("大黄")

print(dog1.kind, dog2.kind)    # 都是犬科
print(dog1.name, dog2.name)    # 旺财 大黄
```

五、类方法与静态方法

```
class Example:
    count = 0
```

```
def __init__(self):
    Example.count += 1

@classmethod
def show_count(cls):
    print(f"当前创建了 {cls.count} 个实例")

@staticmethod
def greet():
    print("你好! 这是一个静态方法。")

# 调用
e1 = Example()
e2 = Example()
Example.show_count()
Example.greet()
```

输出:

当前创建了 2 个实例
你好! 这是一个静态方法。

区别总结:

方法类型	关键字	第一个参数	调用方式	典型用途
实例方法	无	self	实例调用	操作实例数据
类方法	@classmethod	cls	类或实例调用	操作类变量
静态方法	@staticmethod	无	类或实例调用	工具函数，不依赖类或实例

六、继承 (Inheritance)

```
class Animal:
    def speak(self):
        print("动物在叫")

class Cat(Animal): # 继承 Animal 类
    def speak(self):
        print("喵喵喵")

cat = Cat()
cat.speak()
```

输出:

喵喵喵

七、魔术方法 (Magic Methods)

魔术方法又叫**特殊方法**，它们以 `__` 开头和结尾（如 `__init__`、`__str__`、`__add__`）。这些方法在特定情况下会被 **Python 自动调用**。

1. `__init__`：初始化方法（构造函数）

```
class Student:
    def __init__(self, name):
        self.name = name
        print("对象已创建")

s = Student("小张")
```

输出：

对象已创建

2. `__str__`：字符串表示

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

    def __str__(self):
        return f"学生姓名: {self.name}, 分数: {self.score}"

s = Student("小李", 90)
print(s)
```

输出：

学生姓名: 小李, 分数: 90

3. `__len__`：定义 `len()` 的行为

```
class MyList:
    def __init__(self, items):
        self.items = items

    def __len__(self):
        return len(self.items)

m1 = MyList([1, 2, 3, 4])
print(len(m1))
```

输出:

```
4
```

4. `__add__`: 定义 `+` 运算符行为

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __str__(self):
        return f"({self.x}, {self.y})"

v1 = Vector(1, 2)
v2 = Vector(3, 4)
print(v1 + v2)
```

输出:

```
(4, 6)
```

5. `__eq__`: 定义 `==` 的比较行为

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

```
def __eq__(self, other):
    return self.x == other.x and self.y == other.y

p1 = Point(2, 3)
p2 = Point(2, 3)
print(p1 == p2)    # True
```

6. `__call__`: 让对象可被“当作函数”调用

```
class Greeter:
    def __init__(self, name):
        self.name = name

    def __call__(self):
        print(f"你好, 我是 {self.name}")

g = Greeter("小美")
g()    # 等价于 g.__call__()
```

输出:

```
你好, 我是 小美
```

7. `__getitem__` 与 `__setitem__`

让自定义对象支持 `[]` 操作。

```
class Box:
    def __init__(self):
        self.items = {}

    def __getitem__(self, key):
        return self.items.get(key)

    def __setitem__(self, key, value):
        self.items[key] = value

box = Box()
box['apple'] = 5
print(box['apple'])
```

输出:

八、常用魔术方法速查表

方法名	触发条件	示例
<code>__init__</code>	创建对象时调用	初始化属性
<code>__str__</code>	<code>print(obj)</code>	返回可读字符串
<code>__len__</code>	<code>len(obj)</code>	返回长度
<code>__add__</code>	<code>obj1 + obj2</code>	定义加法行为
<code>__eq__</code>	<code>obj1 == obj2</code>	定义相等比较
<code>__getitem__</code>	<code>obj[key]</code>	定义索引取值
<code>__setitem__</code>	<code>obj[key] = value</code>	定义索引赋值
<code>__call__</code>	<code>obj()</code>	让对象可被调用
<code>__del__</code>	对象销毁时	清理资源

• re正则表达式

◦ 作用

- 匹配字符串,快速搜索、替换、验证字符串中的特定模式。通过import re模块来实现

◦ 常见函数运用

1. re.match()

- 从字符串**开头**匹配，成功返回匹配成功的值,否则返回None。

```
import re

result= re.match('hello','hello world')
print(result.group())# hello
```

2. re.search()

- 返回**第一个**匹配项

```
result2 = re.search(r'\d+', 'Python 3.10 version')
print(result2.group())# 3
```

3. re.findall()

- 返回所有匹配项并且为列表

```
result3 = re.findall(r'\d+', '1y2ge72hg8723hr87g129dg9712h')
print(result3)#[ '1', '2', '72', '8723', '87', '129', '9712']
result4 = re.findall(r'\d', '1y2ge72hg8723hr87g129dg9712h')
print(result4)#[ '1', '2', '7', '2', '8', '7', '2', '3', '8', '7',
'1', '2', '9', '9', '7', '1', '2']
```

4. re.sub()

- 替换匹配内容

```
x='好ppppp豆pppppppp'
result5 = re.sub(r'p', '', x)
print(result5)#好豆
```

5. re.split()

- 按照匹配的模式分割字符串

```
x='app,ppa/asd|aaa'
result6= re.split(r'[/,|]', x)
print(result6)#[ 'app', 'ppa', 'asd', 'aaa']
```

。 贪婪与非贪婪匹配

- 贪婪: "*"
- 非贪婪: "?"

```
text = "abcdabcdabcd"
greedy = re.findall(r"a.*d", text)
nongreedy = re.findall(r"a.*?d", text)
print(greedy)#[ 'abcdabcdabcd']
print(nongreedy)#[ 'abcd', 'abcd', 'abcd']
```

。 易错点

- 1. 忘记加'r'使用原始字符串
- 2. 默认的贪婪匹配导致过度匹配 改为.*?
- 3. match()不匹配中间内容,只匹配开头,改用research
- 4. .,?,*等字符需要转义,使用.,?,*
- 5. .group()未加

• 常用符号表（作为搜查表）

符号	含义	示例	匹配内容
.	匹配除换行符外的任意字符	a.c	abc, a1c
^	匹配字符串开头	^Hi	"Hi there"
\$	匹配字符串结尾	end\$	"this is the end"
*	匹配前一个字符 0 次或多次	ab*	a, ab, abb
+	匹配前一个字符 1 次或多次	ab+	ab, abb
?	匹配前一个字符 0 次或 1 次	ab?	a, ab
{n}	匹配前一个字符恰好 n 次	a{3}	aaa
{n,}	匹配至少 n 次	a{2,}	aa, aaa, ...
{n,m}	匹配 n 到 m 次	a{1,3}	a, aa, aaa
[]	匹配方括号内任意字符	[abc]	a, b, c
[^]	匹配除方括号内字符外的任意字符	[^abc]	除 a,b,c 以外
\d	匹配数字 (0-9)	\d\d	23
\D	匹配非数字字符		
\w	匹配字母、数字、下划线	\w+	abc_123
\W	匹配非字母数字下划线		
\s	匹配空白字符（空格、制表符）		
\S	匹配非空白字符		
()	分组 (group)	(abc)+	abcabc

• 列表推导式

- [表达式 for 变量 in 可迭代对象 if 条件]

```
squares = [x**2 for x in range(1, 6)]
print(squares)
```

• generator生成器(yield关键字)

◦ 含义

- 是一种返回迭代器的函数：函数中使用**yield**语句返回值并“冻结”函数状态，下次从该处继续执行

◦ 基本用法

- 定义生成器函数：在函数体内使用 yield 返回值。取值方式：for 遍历、next()、或 list()（会消耗完）。

```
def simple_gen():  
    yield 1  
    yield 2  
    yield 3  
  
g = simple_gen()  
print(next(g)) # 1  
print(next(g)) # 2  
for x in g: #由于函数状态冻结，因此前两个值已经被取完，迭代到最后一位  
    print(x) # 3
```

```
###等价写法  
squares = (x*x for x in range(5))  
for v in squares:  
    print(v) # 0 1 4 9 16
```

◦ send的用法

```
def demo():  
    x = yield 'first' # 第一次 yield, 产出 'first'  
    # 当外部 send(value) 恢复时, x 将等于 value  
    yield f'received {x}' # 然后再产出一个字符串  
  
g = demo()  
print(next(g)) # 激活并获得第一次产出: 'first'  
print(g.send(42)) # 发送 42, 函数内部 x=42, 输出 'received 42'  
# 再次发送会导致 StopIteration (生成器结束)
```

◦ 易错点

- 先用一个变量来储存generator()函数，否则会导致generator()不断重复，只输出第一个值

在存档读档中的应用

```
import json

def game_loop():
    # 初始可序列化状态
    state = {"player_hp": 30, "position": 0, "turn": 0}
    # 第一次 yield 返回初始状态，外部可以在开始前读档并 send 进来
    incoming = yield state
    if incoming:
        state = incoming

    # 简单回合循环：玩家移动 -> 敌人攻击，每个动作后都 yield 状态
    while state["player_hp"] > 0 and state["position"] < 5:
        # 玩家移动
        state["position"] += 1
        state["turn"] += 1
        incoming = yield state
        if incoming:
            # 如果收到一个外部送入的 state，就把它作为当前状态并继续循环（实现“读档”）
            state = incoming
            continue

        # 敌人攻击
        state["player_hp"] -= 3
        state["turn"] += 1
        incoming = yield state
        if incoming:
            state = incoming

    # 游戏结束，yield 最后状态然后结束
    yield state

if __name__ == "__main__":
    gen = game_loop()
    state = next(gen) # 必须先 next() 到第一个 yield，才能 send()
    save_slot = None # 内存存档槽（示例）
    print("游戏开始！")

    while True:
        print("当前状态: ", state)
        if state["player_hp"] <= 0:
            print("你已死亡，游戏结束。")
            break
        if state["position"] >= 5:
            print("你到达终点，胜利！")
            break

        cmd = input("输入\n s(保存内存槽)\n l(从内存槽读档)\n f(保存
```

```

到文件)\n o(从文件读    档)\n n(前进一步)\n q(退
出)\n: ").strip().lower()
    if cmd == "s":
        # 内存保存: 注意要 copy, 避免后续 generator 修改同一个
dict
        save_slot = dict(state)
        print("已保存到内存槽。")
    elif cmd == "l":
        if save_slot is None:
            print("内存槽为空。")
        else:
            # 重建 generator 并把保存的状态 send 进去恢复
            gen = game_loop()
            _ = next(gen)                # 先跑到第一个 yield
            state = gen.send(dict(save_slot)) # send 返回下一
个 yield 的值
            print("已从内存槽读档。")
    elif cmd == "f":
        with open("save.json", "w") as f:
            json.dump(state, f)
            print("已保存到 save.json。")
    elif cmd == "o":
        try:
            with open("save.json", "r") as f:
                loaded = json.load(f)
        except FileNotFoundError:
            print("文件不存在。")
            continue
        # 同样要重建 generator 并 send 读取的状态
        gen = game_loop()
        _ = next(gen)
        state = gen.send(loaded)
        print("已从 save.json 读档。")
    elif cmd == "n":
        try:
            state = next(gen) # 继续执行到下一个 yield, 获得新
的状态
        except StopIteration:
            print("游戏逻辑已结束。")
            break
    elif cmd == "q":
        print("退出。")
        break
    else:
        print("未知命令。")

```

◦ Type Hint类型注释

▪ 含义

- 是向代码中添加静态类型信息的一种做法, 它不改变运行时的动态类型行为, 而是为静态分析器和读者提供类型信息, 帮助发现错误、自动补全、文档化代码意图并提

升可维护性。

- 作用

- 1. 运行前提示类型不匹配、参数或返回值错误
- 2. 帮助他人理解函数/类的预期用法

- 函数注释

```
def add(x: int, y: int) -> int:  
    return x + y
```

- 变量注释

```
count: int = 0  
names: list[str] = ["Alice", "Bob"]
```

```
def join_names(names: list[str]) -> str:  
    return ", ".join(names)
```

```
#避免忘记None空值处理 使用Optional  
def greet(name: Optional[str]) -> str:  
    if name is None:  
        return "Hello, Guest"  
    return "Hello, " + name.upper()
```